

Cobat System, a dynamic combat logic plugin for godot

User manual.

A godot plugin to handle combat logic through scripts read out from txt files.

Made using GDExtension C++.

To make easily moddable combat logic in your game.

The new objects

CobatManager

This is the object you need to run the system

You need one of these to run the plugin features. It is the thing that processes things, does functions and centralizes data.

To use it, you place it in your scene, then you just need your games gdscrip to call its features.

Functions:

RunSequence([input])

Runs a script from the text put in.

[input]: the text from a txt file formatted with the scripting features described below.

You can get this by doing something like the following in a script attached to your cobatmanager:

```
var file = FileAccess.open("res://attack.txt", FileAccess.READ)

var content = file.get_as_text()

self.RunSequence(content)
```

ActivateTrigger([input])

Runs through the triggers present on each unit, and everywhere the triggers have the word used as input, they add the name of a txt file to be ran to a queue.

[input]: the word used as input. Which is checked against the word triggers on units have, in order to see which should be added to the queue.

ProcessQueue()

Goes through the list of txt files made by ActivateTrigger, and runs all those files.

AddUnit([input])

Adds a unit to give the CobatManager access to it,

[input]: a pointer to the unit you want to give the CobatManager access to.

CharSheet

The object you use to store data for your unit.

Keeps data and is accessed by your CobatManager.

Functions

InitializeValues([input])

Reads a txt file to construct the data a unit has.

[input]: the text from a txt file containing which variables a unit should have and their starting values.

That txt file can be formatted as:

```
str
name, test
int
hp, 6
damage, 3
speed, 4
```

‘str’ signifies the lines following it are strings.

‘int’ signifies the lines following that are integers.

Then each line has a variable name, like 'name', 'hp' or 'damage'. Followed by ', ' and the value.

You can get this by doing something like the following in a script attached to your cobatmanager:

```
var file = FileAccess.open("res://attack.txt", FileAccess.READ)
var content = file.get_as_text()
self.RunSequence(content)
```

SetName([input])

Sets the name of the unit to what is input.

[input]: the new name.

Basic Syntax

Functions are ran line by line, with the following structure:

```
[function]: [variable1], [variable2]
```

First state the function name, follow that with “: ” (a colon and a space).

If you start the line with “#” or there is no function by the name put here, the line is skipped.

then add variables separated by “, ” (a comma and a space).

Start variables with a single letter indicator for type:

“i” for integers

“s” for strings

So for example, a line can say:

```
divide: i9, i3, sresult
```

Which divides 9 by 3 and then keeps the result for the rest of the time the txt file is being ran under “result”.

Functions

“add”

>add: i7, i4, sresult

Adds two numbers together, and keeps the result for the rest of the txt file being ran.

Inputs:

First integer: the first number to be added.

Second integer: the second number to be added.

First string: the name under which the result is kept.

“subtract”

>subtract: i9, i3, sresult

Subtracts the second given number from the first, and keeps the result for the rest of the txt file being ran.

Inputs:

First integer: the number that we start with.

Second integer: the number that is subtracted from the first.

First string: the name under which the result is kept.

“multiply”

>multiply: i7, i4, sresult

Multiplies two numbers with each other, and keeps the result for the rest of the txt file being ran.

Inputs:

First integer: the first number.

Second integer: the number with which the first is multiplied.

First string: the name under which the result is kept.

“divide”

>divide: i9, i3, sresult

Divides the first number by the second, and keeps the result for the rest of the txt file being ran.

Inputs:

First integer: the first number.

Second integer: the number that the first number is divided by.

First string: the name of the unit of which the variable is to be set.

Second string: the name of the variable to be set.

“addSet”

>addSet: i7, i4, sresult

Adds two numbers together, and sets a numerical variable of a unit to the result.

Inputs:

First integer: the first number to be added.

Second integer: the second number to be added.

First string: the name of the unit of which the variable is to be set.

Second string: the name of the variable to be set.

“subtractSet”

>subtractSet: i9, i3, sresult

Subtracts the second given number from the first, and sets a numerical variable of a unit to the result.

Inputs:

First integer: the number that we start with.

Second integer: the number that is subtracted from the first.

First string: the name of the unit of which the variable is to be set.

Second string: the name of the variable to be set.

“multiplySet”

>multiplySet: i7, i4, sresult

Multiplies two numbers with each other, and sets a numerical variable of a unit to the result.

Inputs:

First integer: the first number.

Second integer: the number with which the first is multiplied.

First string: the name of the unit of which the variable is to be set.

Second string: the name of the variable to be set.

“divideSet”

>divideSet: i9, i3, sresult

Divides the first number by the second, and sets a numerical variable of a unit to the result.

Inputs:

First integer: the first number.

Second integer: the number that the first number is divided by.

First string: the name of the unit of which the variable is to be set.

Second string: the name of the variable to be set.

“makeModifier”

>makeModifier: sPlayer, s<i-damage-i0-i1, s<i-speed-i0-i6, s<s-name-i0-sbanana

Makes a new modifier and adds it to a unit.

Modifiers are used to buff, debuff or otherwise change the stats units use for calculation, without changing the base stat

Inputs:

First string: the name of the unit that the modifier is added to

Second string and subsequent strings: Data the modifier changes, can be any number of entries.

For each entry structure the input as follows: (in order)

“s”: for basic input handling, makes the whole thing be taken by the function as a string.

“<”: this character is there to keep things clear. While running this gets removed, so a character has to be here.

[type specifier]: tells what kind of variable is to be modifies. “i” for integers and “s” for strings.

“-”: a divider, put between the varied parts of an entry.

[name of variable]: the name of the variable the modifier modifies.

[the modifier type]: read like a function input, so needs the ‘i’ as type specifier. Specifies how the modifier modifies the value.

For strings it can only be 0 to append.

For integers:

0: Adds the modifier value to the variable.

1: Subtracts the modifier value from the variable.

2: Multiplies the variable by the modifier value.

3: Divides the variable by the modifier value.

[the modifier value]: with what the modifier adjusts the variable

“trigger”

>trigger: sendTurn

Triggers actions based on trigger conditions present in units. Works ‘ActivateTrigger’ on the CobatManager.

Inputs:

The first string: the word to be triggered.

“makeTrigger”

>makeTrigger: sPlayer, sendTurn, strigger.txt

Makes a new trigger condition and adds it to a unit.

Inputs:

-First string: the name of the unit to add the trigger to.

-Second string: The word on which the trigger activates.

-Third string: the name of text document to be ran when the word is called.

“readout”

Debug function used for early systems testing. Don’t use this one.